



Vue.js 금융 대시보드 개발

3단계 구현 가이드

Vite + Vue 3로 만드는 실무형 SPA 프로젝트 완전 정복

STEP 1



기초 & 라우팅

Vite 프로젝트 생성
Vue Router 설정
임시 로그인 & API 맛보기

STEP 2



UI & 시각화

Vuetify 디자인 적용
Chart.js 시계열 차트
실제 서비스 같은 UI 구현

STEP 3



인증 & 상태관리

Firebase Auth + Firestore
Pinia 스토어 관리
Router Guard & Toast



© 2025 Finance Dashboard Project. All rights reserved.

OVERVIEW

학습 로드맵 & 핵심 원칙

1단계: 기초 구현 & 라우팅

2단계: UI 디자인 & 시각화

3단계: 인증 & 상태관리

비교 요약 및 마무리

1

1단계: 기능 중심 (Minimal) Vite + Router + Axios

금지 항목: Firebase, Pinia, Mitt, Vuetify, Chart.js 오직 라우팅 구조와 임시 로그인 로직, 기본 API 호출에만 집중합니다.

No Auth

No UI Lib

2

2단계: 디자인 & 차트 (Visual) Vuetify + Chart.js

1단계 코드 위에 디자인 시스템과 데이터 시각화를 입힙니다. 인증/저장은 여전히 더미 상태를 유지하며 UI 완성도에 집중합니다.

Vue-Chartjs

Date-fns

3

3단계: 실제 로직 완성 (Real) Firebase + Pinia + Mitt

이제서야 진짜 인증과 전역 상태관리를 도입합니다. Router Guard, Global Toast, Firestore 저장이 추가됩니다.

Auth

Firestore

Store

Introduction

Step 1

Step 2

Step 3

Summary

STEP 1

기초 구현& 라우팅

Vite 프로젝트를 생성하고

Vue Router로 기본 골격을 잡는 단계입니다.

🎯 핵심 목표

Vite + Vue 3 프로젝트 생성

SPA 라우팅 구조 설계

임시 로그인(localStorage) 구현

Axios로 외부 API 호출 맛보기



Vite + Vue 3

최신 프론트엔드 빌드 도구와 Vue 3
Composition API 기반 환경 설정



Vue Router

/login, /register, /dashboard 페이지 이동 및
SPA 구조 이해



Axios API

비동기 데이터 통신 기초 및 외부 금융 데이터 연동
실습



임시 인증

복잡한 백엔드 없이 localStorage를 활용한
간단한 로그인 흐름



1단계 제한 사항 (아직 사용하지 않음)

Firebase

Pinia

Vuetify

Chart.js

mitt

Step 1: 기초

Step 2: UI

Step 3: 인증



STEP 1. SETUP

프로젝트 생성 및 패키지 설치

bash - 80x24

1. Vite 프로젝트 생성 (Vue 3 선택)

```
$npm create vite@latest finance-dashboard
```

✓ Select a framework: Vue

✓ Select a variant: JavaScript

2. 프로젝트 폴더 이동 및 의존성 설치

```
$cd finance-dashboard
```

```
$npm install
```

3. 1단계 핵심 라이브러리 추가

```
$npm install vue-router axios
```

✓ 설치 패키지

vue-router 페이지 이동(라우팅)을 위한 공식 라이브러리

axios 환율 및 코인 정보를 가져올 HTTP 클라이언트

⊘ 아직 설치하지 않음

Firebase, Pinia, Vuetify, Chart.js 등은 2, 3단계에서 순차적으로 설치합니다. 지금은 기본 기능 구현에 집중하세요.

Step 1: 기초

Step 2: UI

Step 3: 인증



main.js (1단계)

```
1 import {createApp} from 'vue'
2 import App from './App.vue'
3 import router from './router' // 1단계: 라우터 불러오기
4
5 import './assets/main.css'
6
7 const app = createApp(App)
8
9 app.use(router) // Vue 인스턴스에 라우터 등록
10
11 app.mount('#app')
```

i 핵심 포인트

Vite 프로젝트 생성 직후에는 라우터가 없으므로, npm install vue-router 후 직접 import하고 app.use()로 등록해야 합니다.

⚠ 주의사항

이 단계에서는 아직 createPinia()나 createVuetify()를 사용하지 않습니다. 가장 기본적인 Vue + Router 구조만 잡습니다.

Step 1: 기초

Step 2: UI

Step 3: 인증



라우터 구성 (기본 버전)

src/router/index.js

```
1 import {createRouter, createWebHistory} from 'vue-router'
2 import LoginView from '../views/LoginView.vue'
3 import RegisterView from '../views/RegisterView.vue'
4 import DashboardView from '../views/DashboardView.vue'
5
6 const routes = [
7   {path: '/', redirect: '/login'},
8   {path: '/login', component: LoginView},
9   {path: '/register', component: RegisterView},
10  {path: '/dashboard', component: DashboardView}
11 ]
12
13 const router = createRouter({
14   history: createWebHistory(),
15   routes
16 })
17
18 export default router
```

1 기본 라우팅 구조

3개의 핵심 뷰(Login, Register, Dashboard)를 등록하고, 루트 경로('/') 접근 시 로그인 페이지로 리다이렉트합니다.

라우트 정의 (Routes)

/login → LoginView

/register → RegisterView

/dashboard → DashboardView



참고: 가드(Guard) 없음

1단계에서는 Navigation Guard를 사용하지 않습니다. 즉, 로그인하지 않아도 URL만 알면 대시보드에 접근 가능한 "장난감" 상태입니다.



App.vue (1단계 - 최소 골격)

src/App.vue

```
1 <template>
2 <divclass="app">
3 <h1class="app-title">Finance Dashboard (Step 1) </h1>
4
5 <navclass="nav">
6 <RouterLinkto="/login">로그인 </RouterLink>
7 <RouterLinkto="/register">회원가입 </RouterLink>
8 <RouterLinkto="/dashboard">대시보드 </RouterLink>
9 </nav>
10
11 <mainclass="main">
12 <RouterView />
13 </main>
14 </div>
15 </template>
16
17 <scriptsetup></script>
18
19 <stylescoped>
20 .app{max-width:960px;margin:0 auto;padding:24px; }
21 .nav{display:flex;gap:12px; }
22 .main{border:1px solid #e5e7eb;border-radius:8px; }
```

Navigation RouterLink 사용

일반 <a> 태그 대신 Vue Router의 <RouterLink>를 사용하여 페이지 새로고침 없는 SPA 네비게이션을 구현합니다.

Rendering RouterView 배치

현재 URL에 매칭되는 컴포넌트(Login, Register, Dashboard 등)가 <RouterView /> 위치에 렌더링됩니다.

Design 최소한의 CSS

1단계에서는 Vuetify 같은 UI 라이브러리 없이, 기본적인 CSS만 사용하여 레이아웃을 잡습니다. (중앙 정렬, 간격 등)

Step 1: 기초

Step 2: UI

Step 3: 인증



LoginView (임시 로그인)



src/views/LoginView.vue

```
LoginView.vue

<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'

const email = ref('')
const password = ref('')

const router = useRouter()

const handleLogin = () => {
  // 1단계: 실제 인증 없음, 바로 대시보드로
  router.push('/dashboard')
}
</script>
```

구현 포인트

1단계에서는 Firebase '로그인 상태'를 흉내 냅니다.
v-model을 사용해 입력값을 받고, 버튼 클릭 시 대시보드로 이동합니다.

의도

"보안"보다는 라우팅 흐름(Router Flow)을 먼저 이해시키는 것이 목적입니다.
진짜 인증은 3단계에서 Firebase로 교체됩니다.

Step 1: 기초

Step 2: UI

Step 3: 인증



```
RegisterView.vue <script setup>

<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'

const email = ref('')
const password = ref('')
const passwordConfirm = ref('')

const router = useRouter()

const handleRegister = () => {
  if (password.value !== passwordConfirm.value) {
    alert('비밀번호가 일치하지 않습니다.')
    return
  }

  // 1단계: 실제 Firebase 등록 없이, 그냥 안내 후 로그인 페이지로 이동
  alert('1단계 데모: 회원가입이 완료되었다고 가정하고 로그인 화면으로 이동합니다.')
  router.push('/login')
}
</script>
```

✓ 목표 확인

실제 백엔드 연동 전, UI 레이아웃과 라우터 이동이 정상적으로 동작하는지 확인하는 것이 목표입니다.

i 참고

Firebase Auth를 이용한 실제 회원가입 로직은 3단계에서 `userStore.register()` 호출로 대체됩니다.



DashboardView (API 호출)

src/views/DashboardView.vue

DashboardView.vue

```

<script setup>
import { ref, onMounted } from 'vue'
import { fetchUsdToKrwAndOthers } from '../services/forexApi'
import { const forexLoading: Ref<boolean> } from '../services/cryptoApi'

const forexLoading = ref(false)
const forexLoading = ref(false)
const forexError = ref('')

const cryptoPrices = ref({})
const cryptoLoading = ref(false)
const cryptoError = ref('')

onMounted(async () => {
  // 환율
  forexLoading.value = true
  try {
    const data = await fetchUsdToKrwAndOthers()
    forexRates.value = data.rates
  } catch (err) {
    forexError.value = '환율 정보를 가져오는데 실패했습니다.'
    console.error(err)
  } finally {
    forexLoading.value = false
  }
}

```

⚡ 1단계 특징

디자인 라이브러리(Vuetify)나 차트 없이, <pre> 태그에 JSON 원본 데이터를 그대로 출력하여 "데이터 연결" 자체에 집중합니다.

☁ Axios 활용

무료 API(Frankfurter, CoinGecko)를 axios.get()으로 호출합니다.
CORS나 인증 키 설정이 필요 없는 오픈 API를 사용하여 실습 복잡도를 낮춥니다.

Step 1: 기초

Step 2: UI

Step 3: 인증